Quality Assurance · UniFECAF

Monitoramento da E-Fecaf Global na Black Friday

Prometheus, Grafana, alertas no Telegram e testes de carga com Locust

falhas no pico de 800 clientes

31%

depois da correção

0%

Diego Silva Araujo
Tutor: Felipe Bonatto · Novembro de 2026

Na última Black Friday, a loja caiu e ninguém sabia por quê

Sem métricas, histórico ou alertas, a equipe ficou às cegas: não sabia onde estava o problema nem quando agir.

Perguntas que ficaram sem resposta

O objetivo do trabalho é respondê-las com dados, antes do próximo evento.

- Quais páginas ficaram lentas?
- O problema era a aplicação ou o banco de dados?
- Com quantos clientes a loja começa a falhar?
- Quando a equipe deveria ter sido avisada?

Os quatro sinais de ouro definiram o que medir

Latência

Tempo de resposta p95: o tempo que 95% dos clientes não ultrapassam

Tráfego

Requisições por segundo, por página da loja

Erros

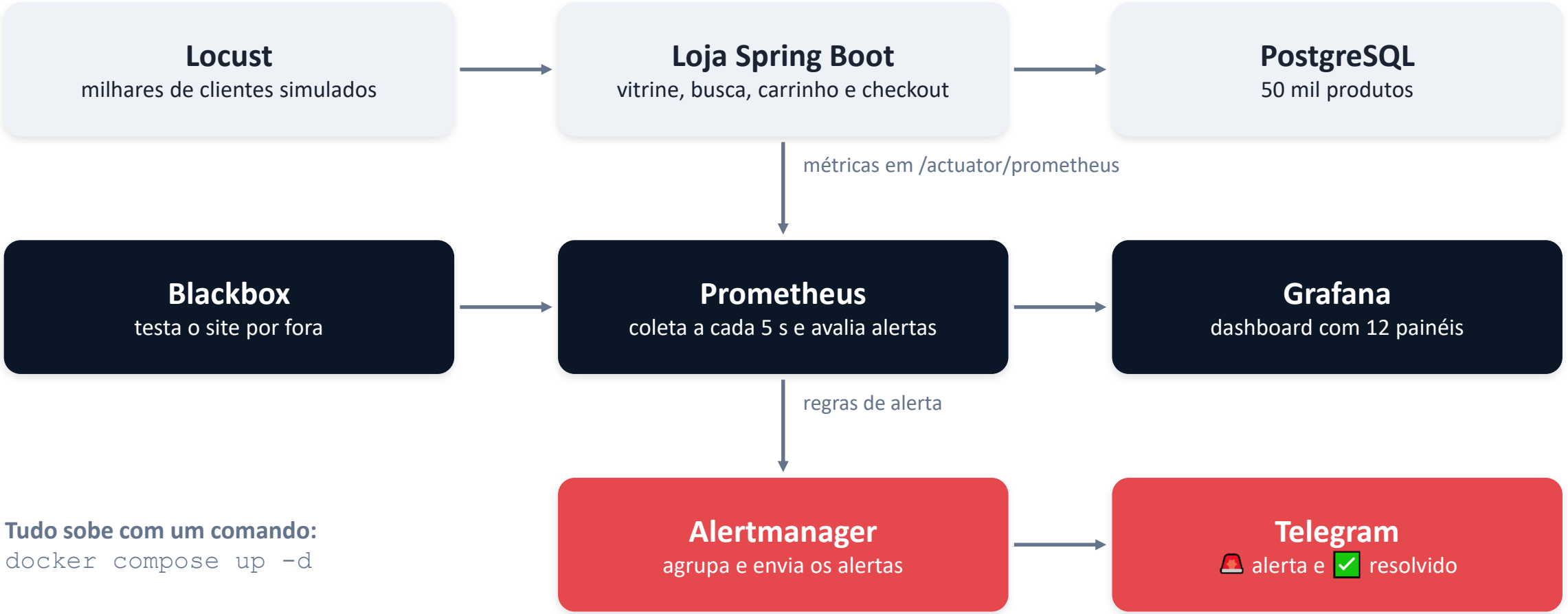
Percentual de respostas 5xx (falhas do servidor)

Saturação

Conexões do banco em uso e na fila, CPU e memória

E mais: disponibilidade medida de fora, como um cliente, pelo Blackbox Exporter.

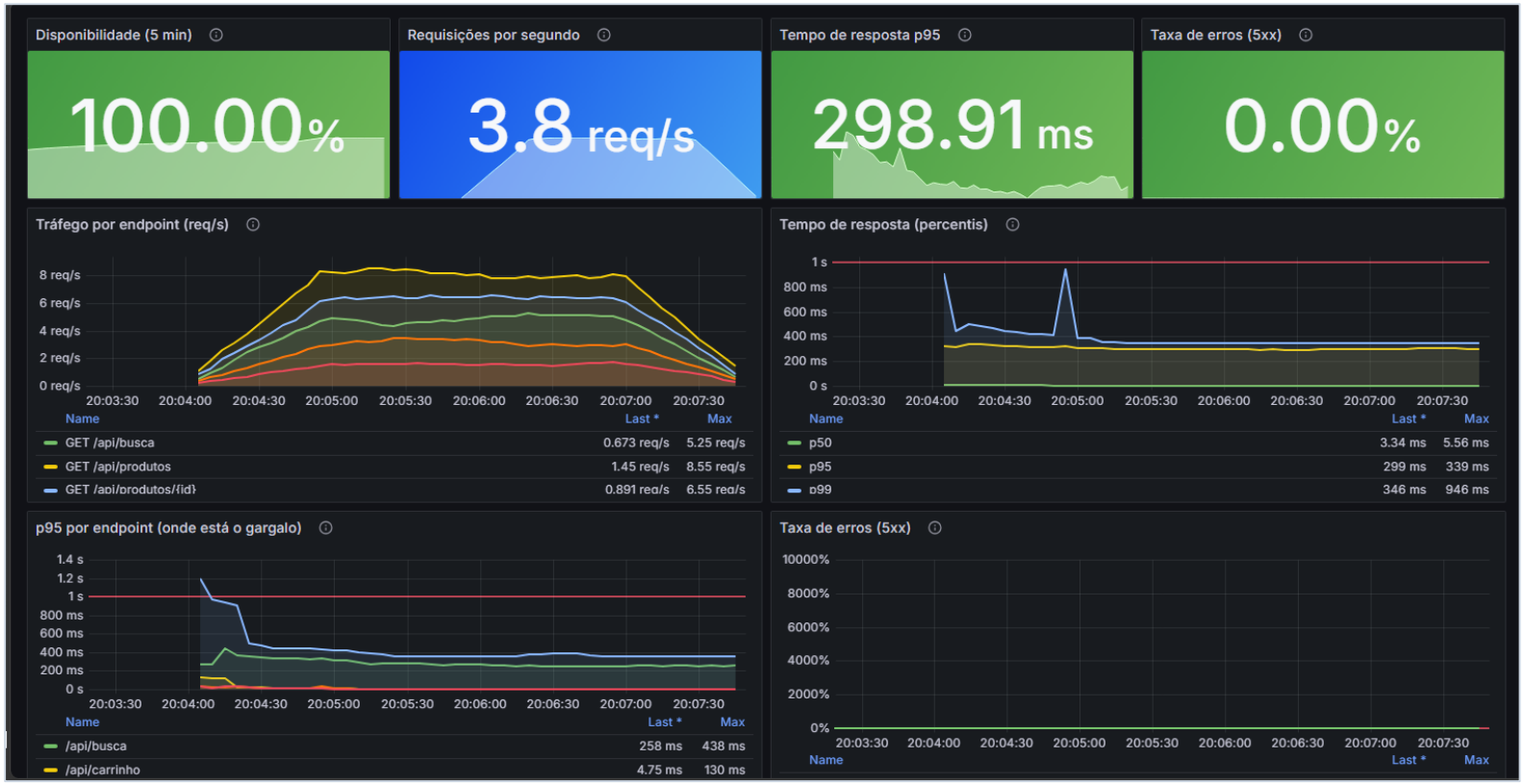
Sete serviços em Docker, do clique do cliente ao alerta no celular



Tudo sobe com um comando:

```
docker compose up -d
```

O Grafana mostra a saúde da plataforma em tempo real



Teste 1 com 50 clientes: 0% de erros e p95 de 0,34 s.

Resumo executivo

Disponibilidade, req/s, p95 e erros mudam de verde para vermelho conforme os limites

p95 por endpoint

Mostra qual parte da loja está lenta: é onde se acha o gargalo

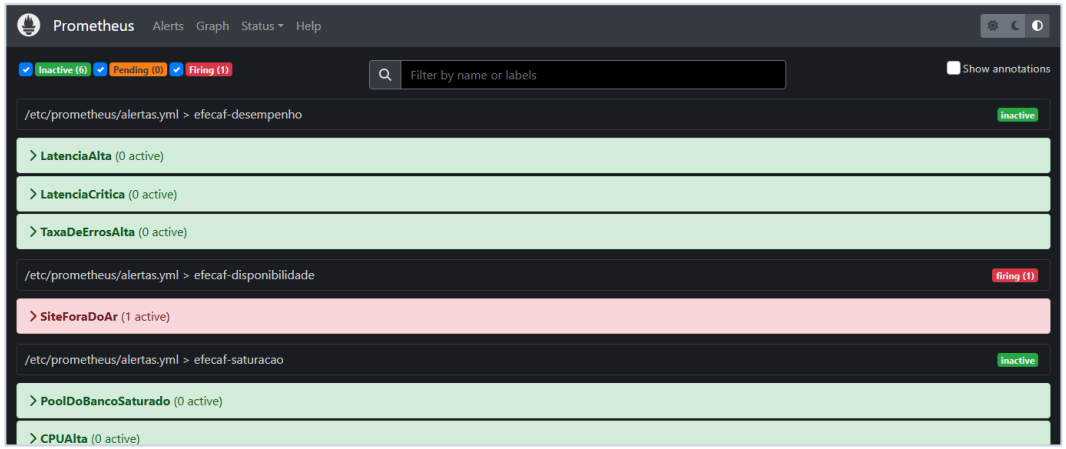
Pool de conexões

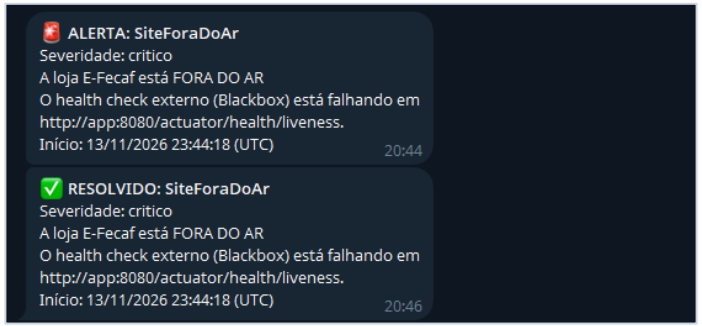
Se a fila sobe, o banco está saturado

Sete alertas avisam a equipe antes de o cliente reclamar

Alerta	Dispara quando	Nível
SiteForaDoAr	o health check externo falha	crítico
LatenciaAlta	p95 acima de 1 s	aviso
LatenciaCritica	p95 acima de 2 s	crítico
TaxaDeErrosAlta	erros 5xx acima de 2%	crítico
PoolDoBancoSaturado	90% das conexões em uso	crítico
CPUAlta	CPU acima de 80%	aviso
MemoriaAlta	memória acima de 85%	aviso

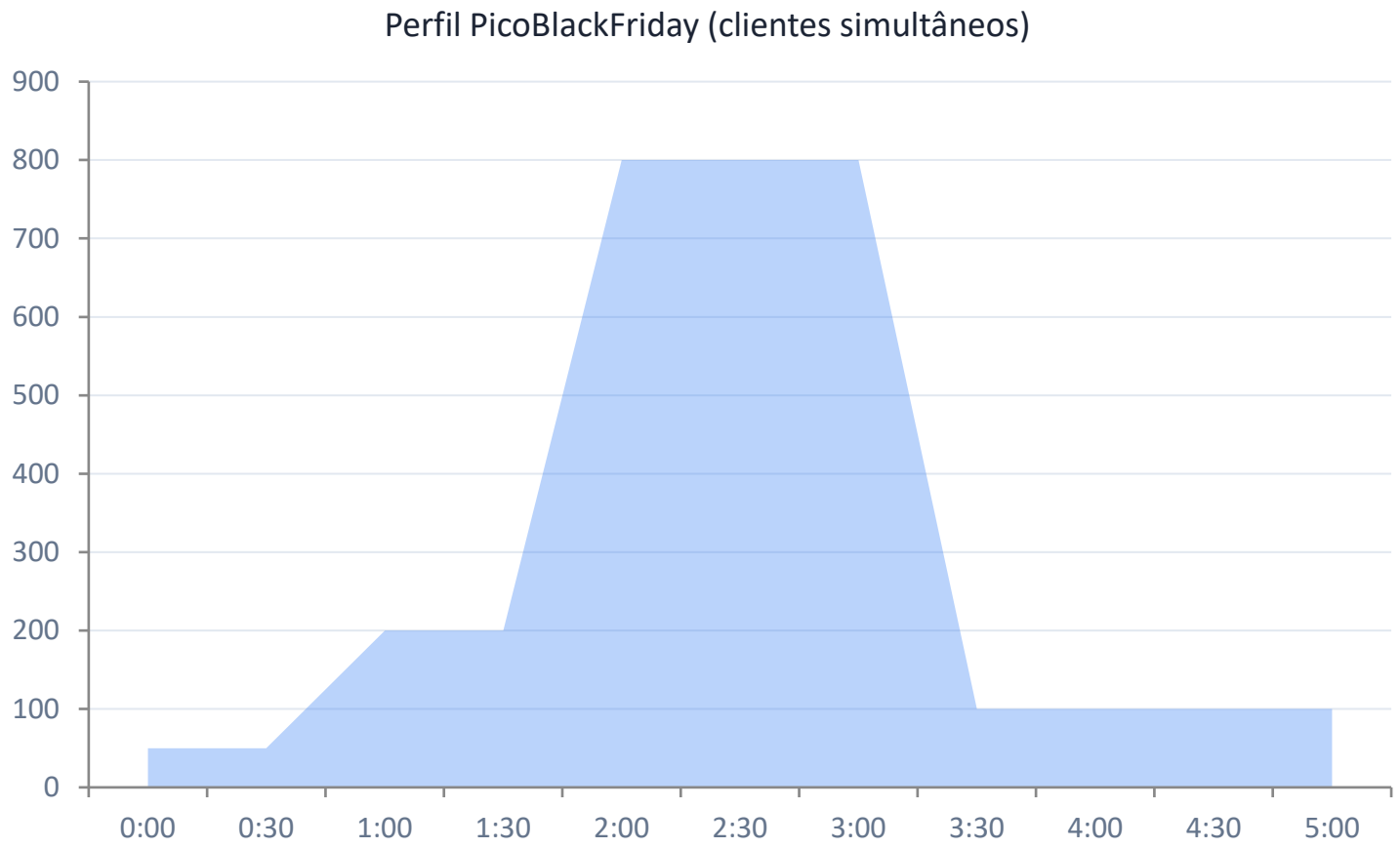
Cada regra só dispara se a condição durar um tempo mínimo (campo for), o que evita alarmes falsos.





Teste de queda: alerta em 30 s e aviso de resolvido 10 s após a loja voltar.

O Locust simulou a Black Friday: de 50 a 800 clientes em 5 minutos



Cada cliente navega, busca, adiciona ao carrinho e compra, com pausas de 1 a 3 s.

Teste 1

Carga normal: 50 clientes por 3 min

Teste 2

Pico de Black Friday (gráfico ao lado)

Teste 3

Estresse: subir até 1.500 clientes

Etapa 8

Otimizar e repetir o Teste 2

No pico, 31% das requisições falharam e 4 alertas dispararam

31%

das requisições falharam

4,8 s

p95 visto pelo cliente

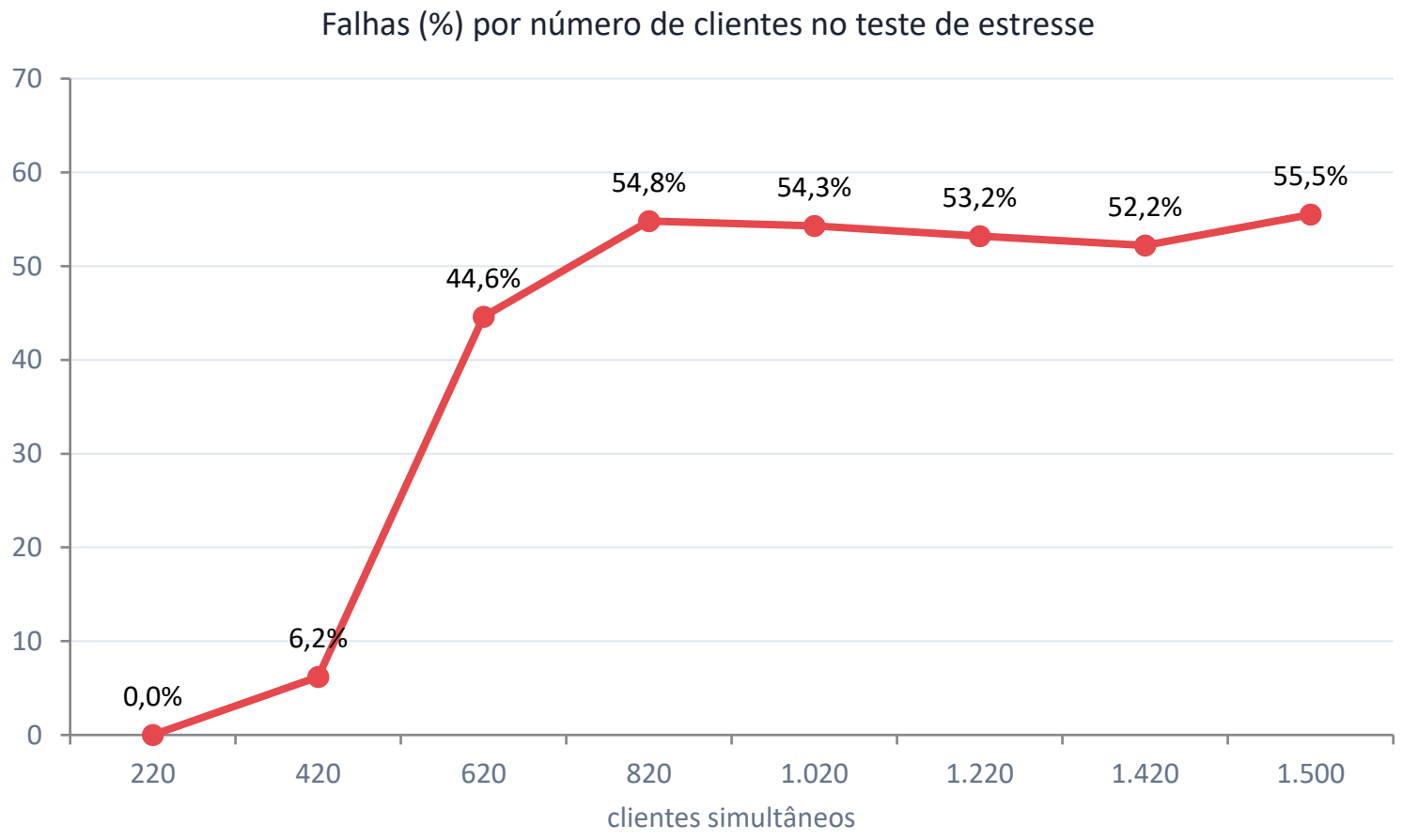
194

requisições esperando conexão com o banco



Grafana durante o Teste 2: p95 acima de 2 s no servidor e taxa de erros chegando a 54% no pico.

A plataforma aguenta entre 220 e 420 clientes simultâneos



420 clientes

erros passam de 5%: é o limite atual

~130 req/s

teto de vazão, não importa quantos clientes entrem

1.500 clientes

até o health check falhou e o alerta SiteForaDoAr disparou

Mais conexões e mais CPU não resolveram: o gargalo era o banco

Primeiro palpite: pool de 20 conexões e CPU dobrada

Falhas no pico

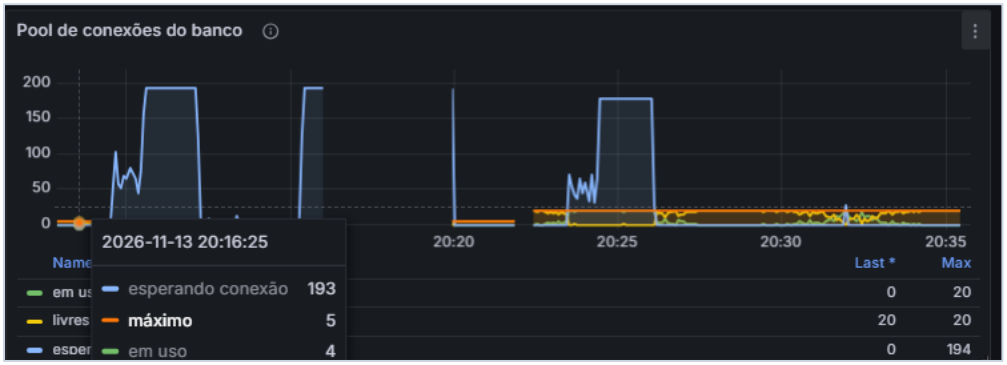
31% → 27%

quase nada mudou

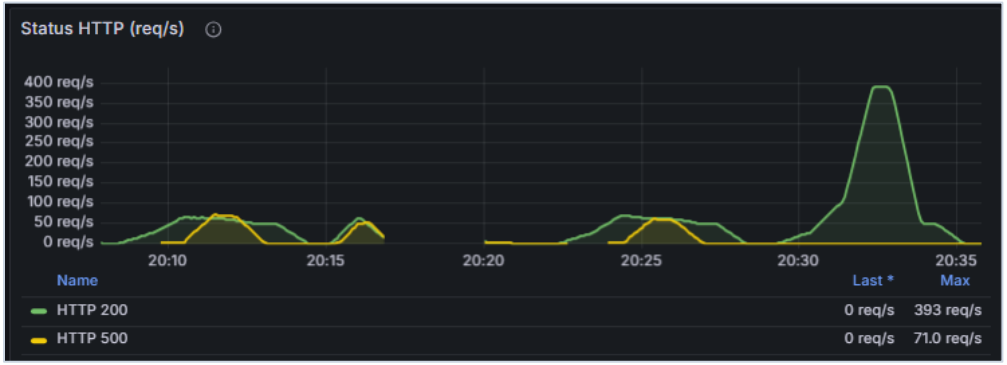
CPU do PostgreSQL

323%

mais de 3 núcleos

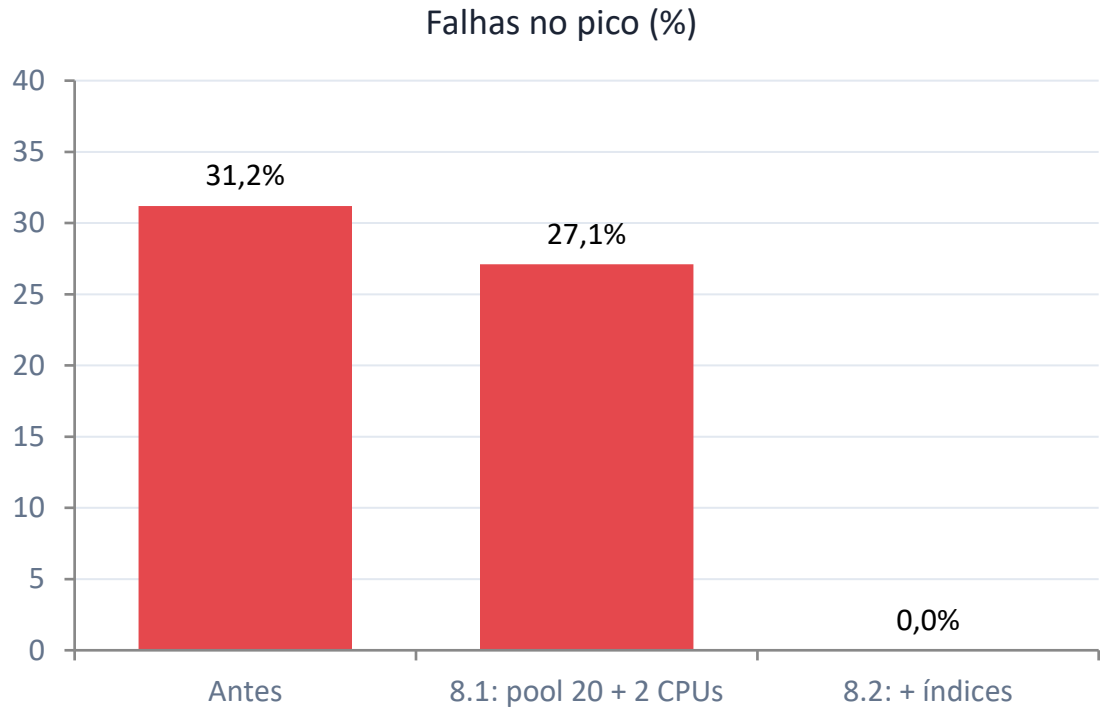


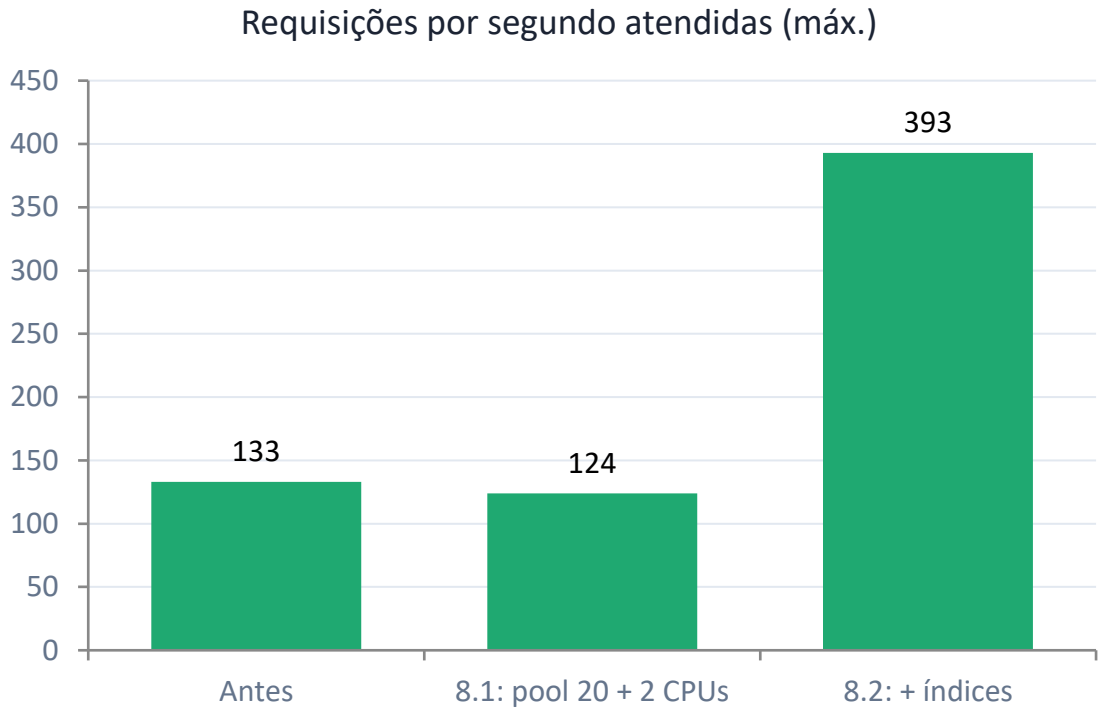
Pool de conexões nos 4 testes: a fila (azul) só some no último, com os índices.



Causa real: a busca não tinha índice. Cada busca lia os 50 mil produtos e gastava 215 ms de CPU. Mais conexões só colocaram mais buscas pesadas ao mesmo tempo no banco.

Com índices na busca, as falhas no pico caíram de 31% para 0%





Busca: **215 ms → 2 ms**

p95 no pico: **4,8 s → 0,31 s**

Capacidade: **2,9× maior**

Alertas: **4 → nenhum**

Próximas otimizações para a próxima Black Friday

01

Pagamento fora da transação

O checkout não segura a conexão do banco enquanto espera o gateway

02

Fila no checkout

Pedidos processados no ritmo que o banco aguenta (RabbitMQ ou Kafka)

03

Cache do catálogo

Vitrine e produtos servidos pelo Redis, sem ir ao banco

04

Réplicas de leitura

Buscas em réplicas; o banco principal fica para os pedidos

05

Mais instâncias

Balanceador de carga e escala horizontal da aplicação

06

Teste antes de cada evento

Rodar o PicoBlackFriday a cada mudança relevante

Sem monitoramento, teríamos comprado servidores. Com ele, achamos a causa

Visibilidade

Painéis e 7 alertas mostram a saúde da loja em tempo real e avisam no Telegram

Limite conhecido

Antes: entre 220 e 420 clientes.
Depois: 800 clientes sem nenhuma falha

Decisão com dados

O palpite óbvio falhou; as métricas apontaram a busca sem índice

Repositório: **github.com/Dieggo775/efecaf-monitoramento**

Obrigado!